

NNN simulator: theory, algorithm, and implementation

Contents

1	Theory, algorithm, and implementation	1
1.1	The lattice gas	1
1.2	Monitoring: the system as an observer sees it	3
1.3	Large deviations	4
1.4	The algorithm	7
1.5	Implementation	13
1.5.1	Configurations as integers	13
1.5.2	Scalability and tests	15
1.6	Parameter reference	18

1 Theory, algorithm, and implementation

These notes accompany the simulator kept in this repository, and follow it from the physics down to the code. Section 1.1 discusses the family of stochastic lattice gases being simulated, fixes the notation, and most of the parameters the simulator takes. Section 1.2 sets up the measurement protocol considered and how it affects the observer’s statistical description of the system. Section 1.3 states the current statistics we are after and carries out the calculation that turns them into an average over trajectories and measurement records. That average is what the simulation estimates: Section 1.4 writes the estimator algorithmically, as a sequence of steps, and Section 1.5 describes how those steps are numerically implemented and how much they cost. Section 1.6 collects every input parameter for reference.

1.1 The lattice gas

We begin with a brief overview of the classical stochastic exclusion processes implemented by the simulator. The system is a chain of L sites, in which particles hop between neighbouring sites subject to the exclusion principle, that forbids double occupancy. The configuration on the chain is thus a binary occupation vector $\mathbf{n} \in \{0, 1\}^L$. The chain may be open, with reservoirs at either end that inject and remove particles, or periodic, with site L connected to site 1; in the latter case, the total particle number is conserved. Figure 1 illustrates the system.

Since particles can only jump left/right to adjacent sites, it is important to define a *bond* as a pair of neighbouring sites $(b, b + 1)$ across which a jumping event occurs. The number of bonds ℓ varies from $\ell = L - 1$ in the open chain to $\ell = L$ on a periodic ring. A right hop moves a particle from b to $b + 1$ at rate $c_{\rightarrow}$ and requires $n_b = 1$, $n_{b+1} = 0$; a left hop is its mirror image at rate $c_{\leftarrow}$. On an open chain four reservoir transitions act on the ends, injecting (removing) particles at rate α (γ) on the left and δ (β) on the right, thus giving reservoir densities $\rho_L = \alpha/(\alpha + \gamma)$ and $\rho_R = \delta/(\delta + \beta)$.

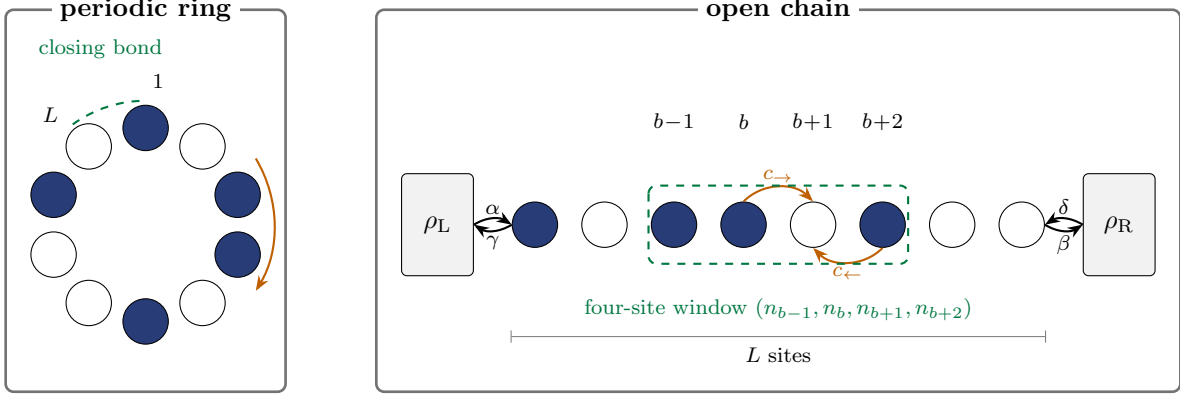


Figure 1: The two geometries; filled circles are occupied sites. On the right, an open chain of L sites between two reservoirs: a particle hops across a bond only into an empty neighbour, at a rate set by its own two sites together with the two flanking them (dashed), while the reservoirs inject and remove at the four boundary rates. On the left, the periodic ring, where site L is joined back to site 1: the reservoirs are absent, the particle number is conserved, and every bond has both of its flanking sites.

We first consider an external field E biasing the hopping. It distinguishes the jump rate in both directions through

$$c_{\rightarrow} = p_0 w, \quad c_{\leftarrow} = q_0 w, \quad p_0 = e^{E/\ell}, \quad q_0 = e^{-E/\ell}, \quad (1)$$

where $w > 0$ is a rate that is independent of the jump direction. A positive E therefore makes a particle more likely to move right than left, a negative one reverses the preference, and $E = 0$ leaves the two directions balanced. Since $p_0 q_0 = 1$, the field fixes only the ratio of the two rates and not their scale. Figure 2 shows the three cases.

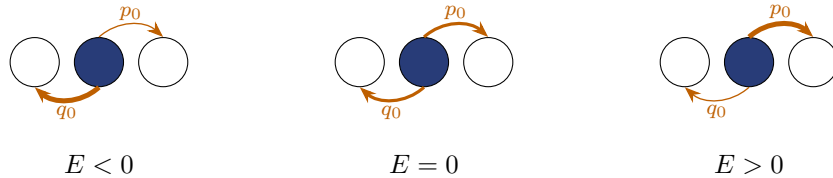


Figure 2: The external field E biases the two hop directions against each other, the thicker arrow marking the faster rate. It enters as $p_0 = e^{E/\ell}$ to the right and $q_0 = e^{-E/\ell}$ to the left, so the product $p_0 q_0 = 1$ is fixed and only the ratio changes with E . The weight w multiplying them is the same in both directions until the interaction of equation 2 is switched on.

In the family of models that we consider, the rate w is not a single number but depends on the occupations of the two sites flanking the hopping pair, that is, on the four-site window $(n_{b-1}, n_b, n_{b+1}, n_{b+2})$. More specifically, the rates become

$$c_{\rightarrow} = p_0 w_{\rightarrow}(n_{b-1}, n_{b+2}), \quad c_{\leftarrow} = q_0 w_{\leftarrow}(n_{b-1}, n_{b+2}). \quad (2)$$

Letting the rate depend on the flanking pair is among the simplest ways of giving the particles an interaction. Equation 2 is the whole family the simulator implements: the eight weights may be handed to it directly, which is the option it calls NNN. Two members of the family are common enough to be worth naming, and are provided as options of their own:

- **WASEP**, the weakly asymmetric simple exclusion process: all eight weights are equal to one, so the flanking sites play no role and the particles are free apart from the exclusion itself. Its open-boundary steady state is known exactly from the matrix ansatz [1], and its current large deviations are the setting in which the additivity principle was formulated [2]. It is therefore the case against which the simulator can be checked.
- **KLS**, the Katz–Lebowitz–Spohn model [3]: the weights are built from two parameters,

$$w_{\rightarrow}(a, d) = 1 + \delta_{\text{KLS}}(1 - a - d) + \epsilon(a - d), \quad w_{\leftarrow}(a, d) = w_{\rightarrow}(d, a), \quad (3)$$

a left hop being the spatial inversion of a right one. ϵ and δ_{KLS} must satisfy $|\epsilon| < 1$ and $|\delta_{\text{KLS}}| < 1$ for every rate to stay positive. A positive ϵ makes the particles repel one another, while δ_{KLS} breaks the symmetry between particles and holes. Setting both to zero returns the WASEP weights.

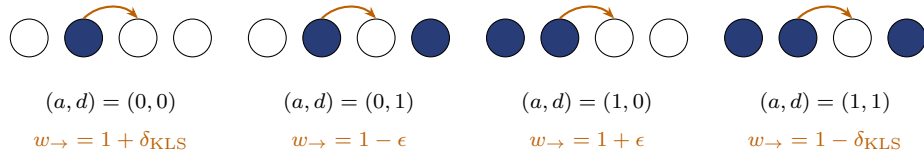


Figure 3: The four rate groups of a right hop; the hopping pair is the middle one, and the two outer circles are the flanking sites. The same hop across the same bond proceeds at four different rates depending only on the occupations of the two sites beside it; the rates shown are those of the Katz–Lebowitz–Spohn model. Left hops are the mirror image, $w_{\leftarrow}(a, d) = w_{\rightarrow}(d, a)$. Setting $\epsilon = \delta_{\text{KLS}} = 0$ makes all four equal and recovers WASEP.

We stress that none of these interaction rates involves the external field, which multiplies them as described above in Eq. (2).

1.2 Monitoring: the system as an observer sees it

The lattice gas of Section 1.1 is now placed under continuous observation. A measurement device is coupled to every site and reports on its occupation through an output signal $\mathcal{S}$, which is what the observer actually sees; Figure 4 sketches the arrangement. How faithfully that signal tracks the true occupation is set by a single parameter k , the measurement strength. For a perfect measurement device that is memoryless, the probability of a given signal, conditioned on a configuration trajectory $\mathcal{C}$, is

$$\mathbb{P}_{[0,t]}(\mathcal{S}|\mathcal{C}) \propto \exp \left[-\frac{k}{2L^2} \int_0^t dt' \sum_{i=1}^L (\mathcal{S}_i(t') - \mathcal{C}_i(t'))^2 \right], \quad (4)$$

where $\mathcal{S}_i(t') \in \mathbb{R}$ is the signal at site i and $\mathcal{C}_i(t') := 2n_i(t') - 1 \in \{-1, 1\}$ is the configuration written in the ± 1 convention. Bayes' rule then inverts this into the observer's statistical description of the system given the recorded signal,

$$\mathbb{P}_{[0,t]}(\mathcal{C}|\mathcal{S}) \propto \mathbb{P}_{[0,t]}(\mathcal{C}) \exp \left[-\frac{k}{2L^2} \int_0^t dt' \sum_{i=1}^L (\mathcal{S}_i(t') - \mathcal{C}_i(t'))^2 \right]. \quad (5)$$

Algorithmically this calls for two objects rather than one. The first is a "physical" realisation of the stochastic process, the trajectory that is actually taking place, on which the measurements are

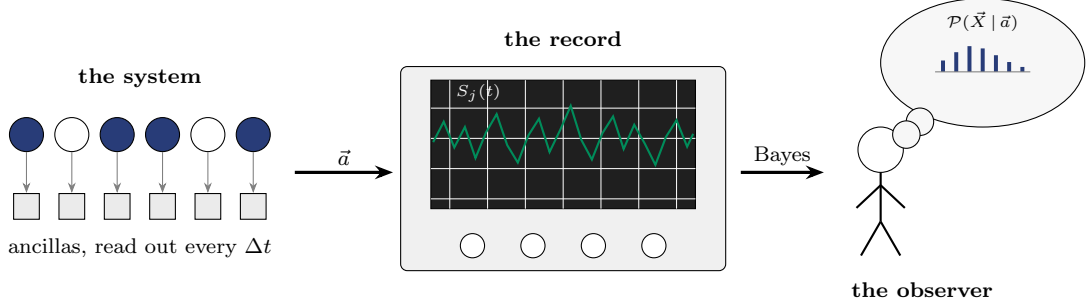


Figure 4: The monitoring, schematically. An ancilla is coupled to every site and read out every Δt , so what leaves the system is not its configuration but a noisy record $\vec{a}$ of it. The observer never sees the chain; they see the record, and from it maintain a probability distribution over configurations, updated by Bayes' rule as each outcome arrives. The population of walkers the simulator evolves is a Monte Carlo representation of exactly that distribution.

performed according to Eq. (4). The second is the observer's description of it, which the simulator represents by a population of walkers, each weighted by its likelihood under Eq. (5) as further outcomes arrive. Denoting the physical trajectory by $\mathcal{C}^{(0)}$, the signal follows the law

$$\mathcal{S}_i(t') = \mathcal{C}_i^{(0)}(t') + \frac{L^2}{k} \frac{dW_i(t')}{dt}, \quad (6)$$

where $dW_i(t')$ is a Wiener process with $\overline{dW_i^2} = \frac{k}{L^2} dt$. Using this result in Eq. (5) gives

$$\mathbb{P}_{[0,t]}(\mathcal{C} | \mathcal{S}) \propto \mathbb{P}_{[0,t]}(\mathcal{C}) \exp \left[\frac{k}{L^2} \int_0^t dt' \sum_{i=1}^L \mathcal{C}_i(t') \mathcal{C}_i^{(0)}(t') + \sum_{i=1}^L \int_0^t dW_i(t') \mathcal{C}_i(t') \right]. \quad (7)$$

1.3 Large deviations

Having specified the dynamics and the measurement protocol, we can now turn to the observable whose fluctuations the simulator is designed to resolve: the particle current. Recall that ℓ is the number of bonds, with $\ell = L - 1$ for an open chain and $\ell = L$ for a periodic ring. For each bond b , let $Q_b(t)$ denote the net number of particles that have crossed it by time t , counting rightward and leftward passages as $+1$ and -1 , respectively, and let $J_b(t) = Q_b(t)/t$ be the corresponding time-averaged current. Particle conservation bounds $Q_b(t) - Q_{b'}(t)$ by the change in particle number in the segment between the two bonds. This difference cannot grow with time, and therefore $J_b(t) - J_{b'}(t) \rightarrow 0$ as $t \rightarrow \infty$. We may thus average over bonds without changing the long-time current:

$$J(t) := \frac{1}{\ell} \sum_{b=1}^{\ell} J_b(t) = \frac{1}{t\ell} \sum_{b=1}^{\ell} Q_b(t) =: \frac{\mathcal{Q}_t[\mathcal{C}]}{t}. \quad (8)$$

Although the bond dependence disappears from the long-time current, $J(t)$ remains a random variable that differs between realisations of the dynamics. Its distribution concentrates around the mean at long times, while an atypical value j is exponentially suppressed: $\mathbb{P}(J(t) \simeq j) \asymp e^{-tI(j)}$, with I the rate function. All of that distribution, its rare values included, is carried by the scaled cumulant generating function

$$\chi(s) = \lim_{t \rightarrow \infty} \frac{1}{t} \log \left\langle e^{s t J(t)} \right\rangle, \quad (9)$$

where $\langle \cdot \rangle$ denotes an average over process trajectories. Indeed, it is well-known from large-deviation theory [4] that χ is the Legendre transform of I . Its derivatives at $s = 0$ give the current cumulants: the mean at first order and the variance at second order. A nonzero s probes the tails of the distribution: positive values favour trajectories with an atypically large current, whereas negative values favour those with an atypically small current.

To evaluate this average, we write the integrated bond-averaged current as the trajectory functional

$$\mathcal{Q}_t[\mathcal{C}] = \sum_{m=1}^{M_t} q(\mathcal{C}^{(m-1)} \rightarrow \mathcal{C}^{(m)}), \quad (10)$$

Here M_t is the total number of transitions by time t , and $\mathcal{C}^{(m)}$ is the configuration immediately after the m th transition. Each bulk hop contributes $+1/\ell$ to $\mathcal{Q}_t$ if it is to the right and $-1/\ell$ if it is to the left; by choice, reservoir transitions are not counted in the open case. We shall now rewrite this average in a form suitable for simulation. For a Markov jump process, the probability of a trajectory factorises into contributions from its jumps and the waiting times between them:

$$\mathbb{P}_{[0,t]}(\mathcal{C}) = \prod_{m=1}^{M_t} W(\mathcal{C}^{(m-1)} \rightarrow \mathcal{C}^{(m)}) \exp \left[- \int_0^t dt' R(\mathcal{C}(t')) \right], \quad R(\mathcal{C}) = \sum_{\mathcal{C}'} W(\mathcal{C} \rightarrow \mathcal{C}'), \quad (11)$$

Here $R(\mathcal{C})$ is the escape rate from the configuration $\mathcal{C}$. Multiplication by $e^{s\mathcal{Q}_t[\mathcal{C}]}$ attaches a factor $e^{sq(\mathcal{C} \rightarrow \mathcal{C}')}$ to each bulk transition. We absorb these factors into the *tilted* rates,

$$W_s(\mathcal{C} \rightarrow \mathcal{C}') = W(\mathcal{C} \rightarrow \mathcal{C}') e^{sq(\mathcal{C} \rightarrow \mathcal{C}')}. \quad (12)$$

For the bulk rates, where $q = \pm 1/\ell$, this factor is equivalent to replacing the field E by $E + s$. Equation 2 therefore becomes $w_{\rightarrow}(a, d) p_s$ and $w_{\leftarrow}(a, d) q_s$, with $p_s = e^{(E+s)/\ell}$ and $q_s = e^{-(E+s)/\ell}$. The reservoir rates remain unchanged because, by choice, reservoir transitions do not contribute to the current and hence have $q = 0$. The tilted rates define a new escape rate $R_s(\mathcal{C}) = \sum_{\mathcal{C}'} W_s(\mathcal{C} \rightarrow \mathcal{C}')$. Because the waiting-time factor in $\mathbb{P}_{[0,t]}(\mathcal{C})$ still contains the original escape rate R , changing the jump rates from W to W_s leaves a correction along the trajectory that we call the counting-field potential,

$$V_s(\mathcal{C}) := R_s(\mathcal{C}) - R(\mathcal{C}).$$

The change of trajectory measure is then

$$\mathbb{P}_{[0,t]}^{(s)}(\mathcal{C}) e^{s\mathcal{Q}_t[\mathcal{C}]} = \mathbb{P}_{[0,t]}^{(s)}(\mathcal{C}) \exp \left[\int_0^t dt' V_s(\mathcal{C}(t')) \right], \quad (13)$$

Here $\mathbb{P}_{[0,t]}^{(s)}$ is the trajectory probability of the Markov process generated by W_s . Equation 13 therefore separates the bias into two parts: the modified dynamics and the escape-rate correction accumulated along each trajectory. Averaging this identity and taking the long-time limit gives the unmonitored CGF,

$$\chi_0(s) = \lim_{t \rightarrow \infty} \frac{1}{t} \log \left\langle \exp \left[\int_0^t dt' V_s(\mathcal{C}(t')) \right] \right\rangle_s, \quad (14)$$

where $\langle \cdot \rangle_s$ denotes an average over those tilted dynamics.

To include measurements, we use equation 7 directly. The physical trajectory $\mathcal{C}^{(0)}$ and the Wiener process W together specify the measurement record, and they assign each candidate trajectory $\mathcal{C}$ the additional weight $e^{\mathcal{M}_{\mathcal{C}^{(0)}, W}[\mathcal{C}]}$, where

$$\mathcal{M}_{\mathcal{C}^{(0)}, W}[\mathcal{C}] := \frac{k}{L^2} \int_0^t dt' \sum_{i=1}^L \mathcal{C}_i(t') \mathcal{C}_i^{(0)}(t') + \sum_{i=1}^L \int_0^t \mathcal{C}_i(t') dW_i(t').$$

Combining this measurement weight with the change of measure in equation 13, and normalising by the corresponding untilted conditional weight, gives the monitored CGF directly as

$$\chi_k(s) = \lim_{t \rightarrow \infty} \frac{1}{t} \mathbb{E}_{\mathcal{C}^{(0)}, W} \left[\log \frac{\left\langle \exp \left[\int_0^t dt' V_s(\mathcal{C}(t')) + \mathcal{M}_{\mathcal{C}^{(0)}, W}[\mathcal{C}] \right] \right\rangle_s}{\left\langle e^{\mathcal{M}_{\mathcal{C}^{(0)}, W}[\mathcal{C}]} \right\rangle_0} \right]. \quad (15)$$

Here the outer expectation averages over the physical trajectory $\mathcal{C}^{(0)}$ and the Wiener process W with the laws specified in Section 1.2; the denominator inside the logarithm is averaged over the original, untilted dynamics. The Itô integral in $\mathcal{M}_{\mathcal{C}^{(0)}, W}$ typically grows as $\sqrt{t}$, whereas the other terms in the exponent grow linearly with t . Its contribution to the logarithm is therefore sublinear and disappears after division by t in the long-time limit. As a consequence, removing the Itô term from both the numerator and its normalisation leaves the measurement potential

$$U_k(\mathcal{C}, t) := \frac{k}{L^2} \sum_{i=1}^L \mathcal{C}_i \mathcal{C}_i^{(0)}(t).$$

Together with the counting-field potential, it defines the total potential

$$\Phi_{s,k}(\mathcal{C}, t) := V_s(\mathcal{C}) + U_k(\mathcal{C}, t).$$

The monitored CGF can then be written as

$$\chi_k(s) = \lim_{t \rightarrow \infty} \frac{1}{t} \mathbb{E}_{\mathcal{C}^{(0)}} \left[\log \frac{\left\langle \exp \left[\int_0^t dt' \Phi_{s,k}(\mathcal{C}(t'), t') \right] \right\rangle_s}{\left\langle \exp \left[\int_0^t dt' \Phi_{0,k}(\mathcal{C}(t'), t') \right] \right\rangle_0} \right], \quad (16)$$

Here $\Phi_{0,k} = U_k$ because $V_0 = 0$. The numerator and denominator correspond to two evolutions that we can simulate, at s and $s = 0$, respectively. When $k = 0$, the measurement potential and its normalisation vanish, recovering equation 14.

Rather than waiting until the end of a trajectory to evaluate the averages in equation 16, we can update the trajectory weights as the dynamics proceeds. For a fixed physical trajectory $\mathcal{C}^{(0)}$, let $\tilde{P}_s(\mathcal{C}, t | \mathcal{C}^{(0)})$ be the total weight of all tilted trajectories that end in configuration $\mathcal{C}$ at time t :

$$\tilde{P}_s(\mathcal{C}, t | \mathcal{C}^{(0)}) := \left\langle \mathbf{1}_{\{\mathcal{C}'(t)=\mathcal{C}\}} \exp \left[\int_0^t d\tau \Phi_{s,k}(\mathcal{C}'(\tau), \tau) \right] \right\rangle_s, \quad Z_s(t | \mathcal{C}^{(0)}) := \sum_{\mathcal{C}} \tilde{P}_s(\mathcal{C}, t | \mathcal{C}^{(0)}). \quad (17)$$

Here $\mathcal{C}'$ is the trajectory being averaged over. Summing over the final configuration removes the endpoint condition, so Z_s is precisely the numerator of equation 16; at $s = 0$, the same expression gives its denominator. Since $\tilde{P}_s$ does not generally sum to one, we divide it by Z_s to obtain the probability distribution

$$P_s(\mathcal{C}, t | \mathcal{C}^{(0)}) := \frac{\tilde{P}_s(\mathcal{C}, t | \mathcal{C}^{(0)})}{Z_s(t | \mathcal{C}^{(0)})}.$$

Thus $\sum_{\mathcal{C}} P_s(\mathcal{C}, t | \mathcal{C}^{(0)}) = 1$ for every s . For a generic observable A , write the average

$$\langle A \rangle_s(t) := \sum_{\mathcal{C}} A(\mathcal{C}, t) P_s(\mathcal{C}, t | \mathcal{C}^{(0)}).$$

The probabilities $P_s(\mathcal{C}, t \mid \mathcal{C}^{(0)})$ evolve according to

$$\begin{aligned} \partial_t P_s(\mathcal{C}, t \mid \mathcal{C}^{(0)}) &= \sum_{\mathcal{C}' \neq \mathcal{C}} W_s(\mathcal{C}' \rightarrow \mathcal{C}) P_s(\mathcal{C}', t \mid \mathcal{C}^{(0)}) - R_s(\mathcal{C}) P_s(\mathcal{C}, t \mid \mathcal{C}^{(0)}) \\ &+ [\Phi_{s,k}(\mathcal{C}, t) - \langle \Phi_{s,k} \rangle_s(t)] P_s(\mathcal{C}, t \mid \mathcal{C}^{(0)}), \end{aligned} \quad (18)$$

where the chosen initial distribution fixes $P_s(\mathcal{C}, 0 \mid \mathcal{C}^{(0)}) = P_{\text{ini}}(\mathcal{C})$. The first line describes the usual probability flow generated by the tilted rates W_s , while the second redistributes probability towards configurations with a larger $\Phi_{s,k}$. Subtracting the average of $\Phi_{s,k}$ ensures that the probabilities continue to sum to one, and the corresponding overall factor is recorded in Z_s :

$$\frac{d}{dt} \log Z_s(t \mid \mathcal{C}^{(0)}) = \langle \Phi_{s,k} \rangle_s(t), \quad Z_s(t \mid \mathcal{C}^{(0)}) = \exp \left[\int_0^t dt' \langle \Phi_{s,k} \rangle_s(t') \right].$$

Using Z_s for the numerator and Z_0 for the denominator, equation 16 becomes

$$\begin{aligned} \chi_k(s) &= \lim_{t \rightarrow \infty} \frac{1}{t} \mathbb{E}_{\mathcal{C}^{(0)}} \left[\log Z_s(t \mid \mathcal{C}^{(0)}) - \log Z_0(t \mid \mathcal{C}^{(0)}) \right] \\ &= \lim_{t \rightarrow \infty} \frac{1}{t} \mathbb{E}_{\mathcal{C}^{(0)}} \left[\int_0^t dt' (\langle \Phi_{s,k} \rangle_s(t') - \langle \Phi_{0,k} \rangle_0(t')) \right]. \end{aligned} \quad (19)$$

The two equalities in equation 19 lead to independent numerical estimates of the same growth rate: one from $\log Z_s/t$ and the other from the time average of $\langle \Phi_{s,k} \rangle_s$. Their agreement provides a useful cross-check. The monitored CGF is obtained only after subtracting the corresponding result at $s = 0$.

Finally, there exists a third route to the CGF that uses its derivative. Differentiating its defining logarithm, Eq. (9), with respect to s brings down the integrated current $\mathcal{Q}_t$. Therefore

$$\chi'_k(s) = \lim_{t \rightarrow \infty} \frac{1}{t} \mathbb{E}_{\mathcal{C}^{(0)}} [\langle \mathcal{Q}_t \rangle_s], \quad \chi_k(s) = \int_0^s ds' \chi'_k(s'). \quad (20)$$

The second identity follows from $\chi_k(0) = 0$, so averaging $\mathcal{Q}_t/t$ at several values of s and integrating over s gives a third estimate of the CGF.

1.4 The algorithm

The algorithm represents the weighted trajectory ensemble by a population of M copies of the system, called *walkers*. Each walker evolves exactly in continuous time under the tilted rates W_s of equation 12, using the Gillespie algorithm [5, 6], and carries the weight ω_i generated by the total potential $\Phi_{s,k}$; the letter ω is kept for this weight alone, the rate weights $w_{\rightarrow}, w_{\leftarrow}$ of equation 2 being a different object. If these weights were allowed to evolve freely, their ratios would grow exponentially with time and the average would soon be dominated by a few walkers. The cloning method [7, 8] controls this growth by introducing cloning times at which the walkers are resampled in proportion to their weights: walkers with large weights are replicated, whereas those with small weights are removed. Resampling keeps the weight spread manageable and directs the population towards the trajectories that contribute more to the weighted distribution. When the cloning times are placed is a matter of efficiency rather than correctness, and how they are found depends on the specific algorithm implementation, so both are settled in the notes below the boxed procedure, once the three implementations we consider are in place.

Written as a sequence of steps, with M the population size and t_{max} the final time, the algorithm is as follows:

1. **Initialise.** Build M walkers in the requested initial configuration, each with weight one, and one reference trajectory (for $k = 0$ the reference trajectory is irrelevant and can be omitted). Set $t = 0$ and the accumulated normalisation $\log Z_s = 0$.
2. **Choose the next stop.**[‡] A *stop* is an instant at which the whole population is brought to a common time and may be examined. It is the earliest of the following possibilities: the next scheduled resampling instant, the next recording time, and $t_{\max}$.
3. **Evolve to the stop.**[†] Walker events must be generated with the tilted rates W_s until the whole population reaches the stop. When a reference trajectory exists, it must be evolved alongside the walkers until the stop, but with the unbiased rates W_0 . We consider three distinct implementations of this step.

Heap. Every process — each walker, and the reference when monitoring is on — holds one pending event time, drawn from its own escape rate $R_s(\mathcal{C})$. The mechanism is named after where those $M + 1$ times are kept: a binary heap, which holds the earliest of them at its root. Then, repeat the following instructions while the earliest pending time lies before the stop:

- (a) Read the root of the heap and advance the global clock to that time.
- (b) Select one possible transition of that walker with probability proportional to its tilted rate, apply it, and update the integrated current by the increment $q(\mathcal{C} \rightarrow \mathcal{C}')$.
- (c) Update the list of allowed transitions locally, around the bond that changed.
- (d) Recompute that process's tilted escape rate $R_s(\mathcal{C})$, the sum of the rates of all transitions now allowed to it, draw its new pending time $t + \text{Exponential}(R_s(\mathcal{C}))$, and sift that one entry back into place in the heap. Every other process keeps the pending event time it already had.

When the earliest pending time falls beyond the stop, advance the global clock to the stop and finish. The whole population is thus at one time at every instant.

Note the advantages of the heap in this implementation: accessing the earliest pending time is $O(1)$, and updating the heap with a new entry takes $O(\log M)$. A flat list would instead have to be scanned in full, at a cost of $O(M)$ per event, to find the same minimum.

Direct. The same events as in **Heap**, but generated one walker at a time instead of in global order. Under monitoring the reference is evolved first, alone, from the current time to the stop, and its events are stored. Then take the walkers one by one, each carrying its own pending time; for one walker, repeat while that time lies before the stop:

- (a) Advance this walker's clock to its pending event time.
- (b) Select one transition with probability proportional to its tilted rate, apply it, and update the integrated current by the increment $q(\mathcal{C} \rightarrow \mathcal{C}')$.
- (c) Update the list of allowed transitions locally, around the bond that changed.
- (d) Recompute this walker's tilted escape rate $R_s(\mathcal{C})$ and draw its new pending time $t + \text{Exponential}(R_s(\mathcal{C}))$.

When the pending time falls beyond the stop, advance this walker's clock to the stop and pass to the next walker. A walker is therefore carried all the way to the stop before the next one starts, and no two walkers share a time until they all reach it.

Uniformized. One clock for the whole population, ticking faster than any configuration can demand, with the excess removed by rejection. With $w_{\max}$ the heaviest of the eight rates of equation 2 and N_b the number of bonds, the proposal rate of one walker and of the population are, respectively,

$$R_{\text{prop}} = (p_s + q_s) w_{\max} N_b + \alpha + \gamma + \delta + \beta, \quad R_{\max} = M R_{\text{prop}}, \quad (21)$$

where the reservoir term is present for an open chain only. When monitoring is on, the reference adds one further process to the same clock, at a rate of the same form (but with $s = 0$). The pending time belongs to the population as a whole; repeat while it lies before the stop:

- (a) Advance the global clock to the pending time.
- (b) Pick one process uniformly among the M walkers, or the reference, in proportion to its share of $R_{\max}$; then, within it, one bond and one direction with probability proportional to p_s and q_s , or one of the four reservoir moves in proportion to its rate.
- (c) Put the proposal through two tests. The first is exclusion: the hop must start from an occupied site and end on an empty one, and a reservoir move must find the boundary site in the state it needs. The second is the rate: the proposal was made at the heaviest rate $w_{\max}$, while the bond it picked can actually have some other rate w , so it is kept with probability $w/w_{\max}$ and dropped otherwise. A proposal that fails either test is discarded, and nothing changes. One that passes both is applied, updating the integrated current by $q(\mathcal{C} \rightarrow \mathcal{C}')$ and the list of allowed transitions locally.
- (d) Draw the new pending time $t + \text{Exponential}(R_{\max})$, which depends on no configuration and therefore never has to be recomputed.

When the pending time falls beyond the stop, advance the global clock to the stop and finish. The rejections are exactly what turns the flat maximal rate back into the correct rate: a transition of weight w is proposed at rate $w_{\max}$ and survives with probability $w/w_{\max}$, so ultimately it occurs at the right rate w .

4. **Accumulate the weights.** Over the same interval, update the weight of each walker independently according to

$$\frac{d}{dt} \log \omega_i(t) = \Phi_{s,k}(\mathcal{C}^{(i)}(t), t). \quad (22)$$

The single numerical weight therefore contains both the counting-field and measurement contributions. Note that this is why the **Direct** construction has to evolve the reference before all the walkers: the measurement contribution compares a walker with the reference at the walker's own time, so the reference events of the whole interval must already be known when each walker is carried across it.

5. **Resample.**[‡] At a resampling stop, replace the weighted population by M equally weighted children, selecting their parents in proportion to the current weights. Before doing so, two matters need to be addressed. First, one may retain a diagnostic to reveal whether the weights have concentrated among too few walkers. A common choice is the effective sample size[°] [9]:

$$\text{ESS} = \frac{(\sum_i \omega_i)^2}{\sum_i \omega_i^2}. \quad (23)$$

Second, update the running estimate of the normalisation. The population mean of the weights at cloning time t_r is the factor by which Z_s has grown since the previous resampling, so

$$\log Z_s \leftarrow \log \left(\frac{1}{M} \sum_i \omega_i \right). \quad (24)$$

Only estimators that depend on the walker weights need to be evaluated at this step, before the weights are discarded. Weight-independent estimators, such as those associated with the second equality in equation 19 and with equation 20, can instead be evaluated at recording times, as described in the next step.

To perform a systematic resampling of the walker population [10, 11], lay the parent weights end to end as intervals of lengths ω_i on a segment of total length $\Omega = \sum_i \omega_i$, draw one uniform offset u , and place M equally spaced marks on the segment:

$$u \sim \text{Uniform}(0, \Omega/M], \quad u_m = u + (m-1) \frac{\Omega}{M}, \quad m = 1, \dots, M, \quad (25)$$

Each mark selects the parent whose interval contains it and produces one identical child. Since interval i occupies a fraction ω_i/Ω of the segment, parent i receives either $\lfloor M\omega_i/\Omega \rfloor$ or $\lceil M\omega_i/\Omega \rceil$ children, with an average of exactly $M\omega_i/\Omega$. A parent whose weight exceeds the mark spacing Ω/M cannot be lost; a lighter parent survives with probability proportional to its weight. Since one random offset determines the entire population, this procedure is cheaper and less noisy than drawing M parents independently. Each child inherits its parent's complete state, including associated attributes such as the total transferred charge Q_t , after which its weight is reset to one.

6. **Record.** If the stop is a recording time, first carry out the resampling procedure of step 5, irrespective of whether the usual resampling criterion is met. This absorbs the accumulated weights into $\log Z_s$ and replaces the weighted population by an equally weighted one. The observables below can then be recorded as unweighted population averages.

Our ultimate quantity of interest is the current CGF $\chi_k(s)$. Section 1.3 established three distinct estimators for it: two based on the growth rate of Z_s , given by the two equalities in equation 19, and a third based on the mean current, given by equation 20. We therefore record the quantities required by each estimator.

As discussed in step 5, the first estimator is obtained from the normalisation increments accumulated at each resampling. At a recording time, write out the current value of

$\log Z_s$, which gives the finite-time growth-rate estimator

$$\hat{g}_k^{(Z)}(s, t) = \frac{1}{t} \sum_{t_r \leq t} \log \left[\frac{1}{M} \sum_i \omega_i(t_r^-) \right] = \frac{\log Z_s(t)}{t}. \quad (26)$$

The second estimator* requires the instantaneous population average of the total potential, so record

$$\bar{\Phi}_{s,k}(t) := \frac{1}{M} \sum_{i=1}^M \Phi_{s,k}(\mathcal{C}^{(i)}(t), t). \quad (27)$$

Its time series yields the second finite-time growth-rate estimator,

$$\hat{g}_k^{(\Phi)}(s, t) = \frac{1}{t} \int_0^t dt' \bar{\Phi}_{s,k}(t'). \quad (28)$$

The third estimator follows from equation 20. For this route, record the population average of the time-averaged current,

$$j_k(s, t) := \frac{1}{Mt} \sum_{i=1}^M Q_t[\mathcal{C}^{(i)}], \quad \lim_{t \rightarrow \infty} j_k(s, t) = \chi'_k(s). \quad (29)$$

Because the population is resampled before recording, its configurations already represent the normalised biased ensemble, and no additional weighting is required. Evaluating $j_k(s, t)$ at several values of s and integrating over s gives the third estimate of $\chi_k(s)$.

Finally, record the population-averaged density profile and, if desired, population diagnostics such as the pre-resampling ESS retained in step 5.

7. Repeat from step 2 until $t = t_{\max}$.

‡ *Cloning-time note.* Steps 2 and 5 leave the schedule of cloning times unspecified. In the actual implementation, it is useful first to define *examination times*, at which the population is brought to a common time and the exact log-weight range is computed:

$$R_\omega := \max_i \log \omega_i - \min_i \log \omega_i. \quad (30)$$

Step 5 is performed only when this range is sufficiently large; otherwise, the evolution continues without resampling. Examination times therefore provide opportunities for cloning without requiring resampling at each one.

Choosing when to examine the population involves a trade-off. Waiting too long allows the weights to concentrate on a few ancestors and increases the variance. Examining too frequently incurs unnecessary population scans, while resampling too frequently repeatedly replaces walkers with copies and reduces their diversity. The engine controls the examination and resampling decisions through separate weight ratios:

- $\text{target}_{\min}$ is the weight ratio at which resampling becomes worthwhile. The population is resampled only when $R_\omega \geq \log(\text{target}_{\min})$; otherwise, evolution continues unchanged.
- $\text{target}_{\max}$ is the weight ratio that must never be exceeded and therefore determines when the population is examined.

Keeping the exact range below $\log(\text{target}_{\max})$ appears to require a full population scan at every instant. A way around this is to track an upper bound on how far the log-weights can spread. From equation 22, $d \log \omega_i / dt = \Phi_{s,k}(\mathcal{C}^{(i)}, t)$, and therefore

$$\frac{dR_\omega}{dt} \leq \max_i \Phi_{s,k}(\mathcal{C}^{(i)}, t) - \min_i \Phi_{s,k}(\mathcal{C}^{(i)}, t). \quad (31)$$

Whether this bound can be used efficiently depends on the event mechanism. Because **Heap** and **uniformized** advance the population in global time order, they can integrate the right-hand side of equation 31 as a running *certificate*. In practice, after each examination, the engine stores the maximum and minimum values of $\Phi_{s,k}$ across the population, updating them whenever an event produces a larger or smaller potential. The stored values can only overestimate and underestimate the instantaneous maximum and minimum, respectively; integrating this conservative envelope may therefore loosen the bound but cannot invalidate it. When the certificate reaches $\log(\text{target}_{\max})$, the population is examined before the true range can exceed that threshold. If the exact range has reached $\log(\text{target}_{\min})$, the population is resampled; otherwise, evolution continues without resampling. In either case, the certificate is reset to the exact post-examination range.

Direct cannot maintain a population-wide bound between stops because its walkers reach a common time only at a stop. It therefore predicts the next stop using an adaptive time step τ :

$$\begin{aligned} \text{stop} &= \min(t + \tau, \text{next recording}, t_{\max}), \\ \text{initial step: } \tau &= \log(\text{target}_{\max}) / \text{widest possible spread of } \Phi_{s,k}, \\ \text{at the stop: } R_\omega &= \text{exact log-weight range}, \quad r = R_\omega / \text{elapsed time}, \\ R_\omega > \log(\text{target}_{\max}) &\Rightarrow \text{restore the previous state, } \tau \leftarrow \tau/2, \text{ repeat the interval,} \\ \text{otherwise} &\Rightarrow \text{accept the evolution, } \tau \leftarrow \min(\log(\text{target}_{\min})/r, 2\tau). \end{aligned} \quad (32)$$

For the first interval, τ is chosen from the widest potential spread permitted by the model, ensuring that R_ω cannot reach $\log(\text{target}_{\max})$ before the predicted stop. Subsequent steps use the measured growth rate of R_ω to estimate the next time step, which is capped at twice the previous value. Before each interval, the population, reference, and random-number generator are snapshotted. If R_ω exceeds $\log(\text{target}_{\max})$ at the end of the interval, the previous state is restored and the interval is replayed with $\tau/2$, leaving no trace of the rejected attempt. Recording stops undergo the same check, ensuring that no row is written after an overshoot.

† *Mechanism note.* The three mechanisms in step 3 are implemented as separate classes in `Simulator/cpp/engine.cpp`. All generate the same dynamics, but at different speeds. Each provides both a general weighted loop and a specialised loop for $s = k = 0$. In this unbiased case, every weight remains exactly one and cloning has no effect, so the engine bypasses the weights, bounds, resampling, and their associated memory, recording the ordinary trajectory ensemble directly. Out of the three mechanisms, **Direct** is the fastest: the measurements in Section 1.5.2 show speedups of 1.2–2.8 over **heap** and 2.7–5.1 over **uniformized**, depending on the model and the size. Because walkers evolve independently between stops, the **Direct** mechanism can process each walker continuously over the entire interval. This avoids maintaining a global event ordering and improves data locality. The absence of a shared clock, however, slightly complicates the treatment of cloning times: examination times must be predicted adaptively rather than triggered by a running population-wide certificate, as described in the cloning-time note above. **Uniformized** is the only mechanism that is not rejection-free; its rejection rate increases as the transition rates become more heterogeneous, making it the slowest. Its simplicity and independent construction nevertheless make it valuable for validating the other two implementations.

◦ *Diagnostic note.* In addition to the ESS, we record the maximum instantaneous deviation of the total potential from its population mean:

$$\Delta\Phi(t) := \max_i \left| \Phi_{s,k}(\mathcal{C}^{(i)}(t), t) - \overline{\Phi}_{s,k}(t) \right|. \quad (33)$$

Both quantities are optional population diagnostics: they do not estimate an observable, but help assess the reliability of the recorded results. The ESS measures how many walkers contribute effectively to the population average. It equals M when all weights are equal and approaches one as the weights concentrate on a few ancestors. A collapsed ESS therefore means that the average is based on far fewer independent trajectories than the nominal population size M suggests. The potential spread helps diagnose the cause of this loss of effective population size. A broad range of potentials allows the log-weights to separate rapidly, as quantified by the bound in equation 31. The ESS and potential spread are convenient choices rather than a canonical pair. Other useful diagnostics may also be recorded, including the number of distinct ancestors surviving a resampling, the fraction of walkers removed, and the variance of the log-weights.

* *CGF-estimator note.* Both $\hat{g}_k^{(Z)}$ and $\hat{g}_k^{(\Phi)}$ estimate the growth rate of Z_s , rather than the CGF itself. The corresponding CGF estimates are obtained by subtracting the result of an $s = 0$ run at the same value of k :

$$\hat{\chi}_k^{(a)}(s, t) = \hat{g}_k^{(a)}(s, t) - \hat{g}_k^{(a)}(0, t), \quad a \in \{Z, \Phi\}.$$

Agreement between these two estimates provides an internal consistency check.

Figure 5 summarizes the algorithmic loop using the colour scheme introduced above. Black denotes the steps shared by all three implementations, while the coloured annotations distinguish the three event-generation mechanisms described in step 3.

1.5 Implementation

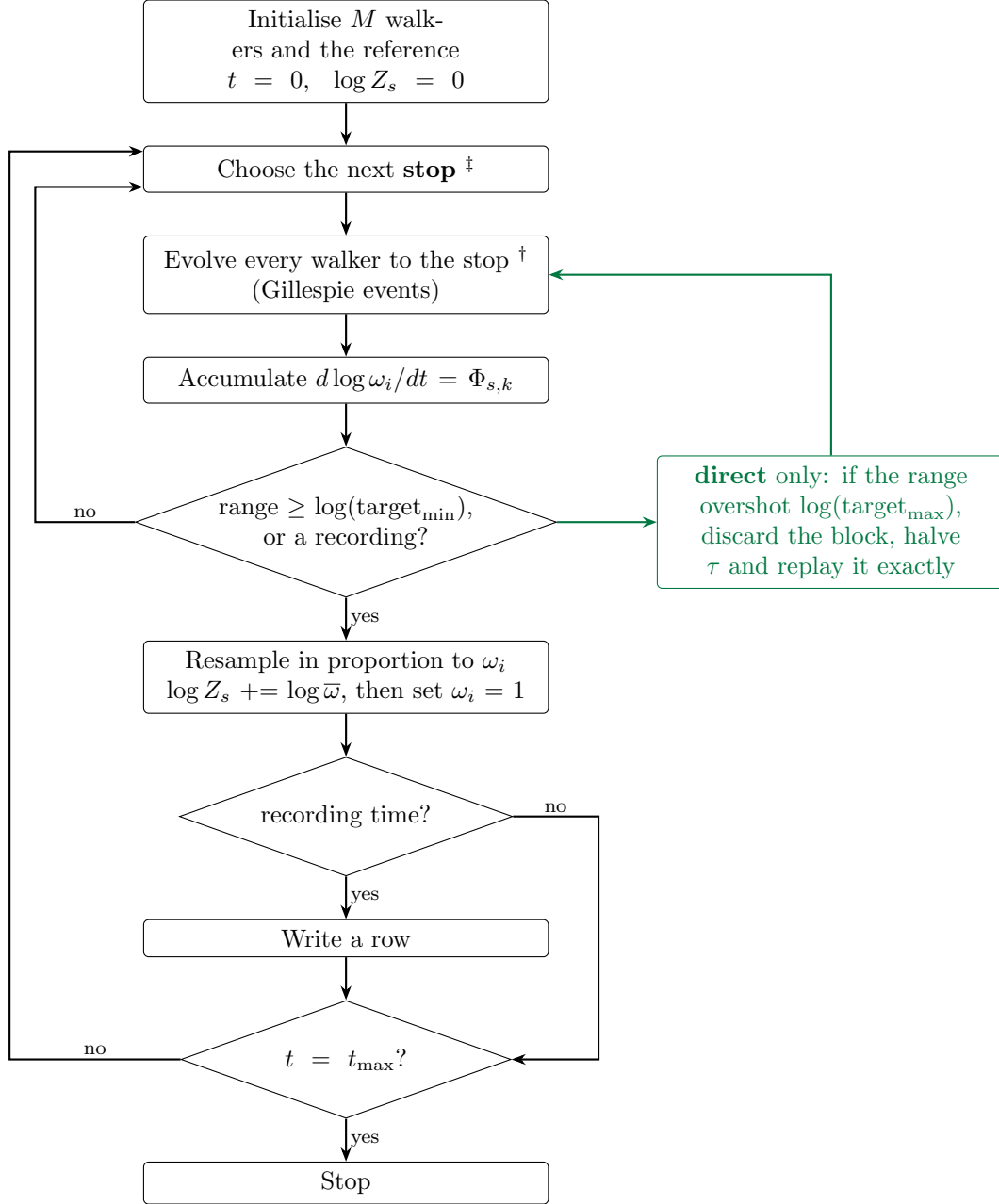
The implementation relies on the specific model family introduced in Section 1.1. The two sites flanking a hopping pair have only four possible occupation patterns, so every allowed bond belongs to one of four *rate groups* in each direction. All bonds within a group share the same rate, as shown in Figure 3. This degeneracy enables the selection scheme below: a single uniform draw can select both a group and, by division, a bond within it.

1.5.1 Configurations as integers

A configuration is a binary vector and is stored as an array of $\lceil L/64 \rceil$ 64-bit words, with site i occupying bit $i \bmod 64$ of word $\lfloor i/64 \rfloor$. Although this representation is not required by the physics, it substantially reduces the cost of the inner loop: operations on a configuration become machine instructions on entire words rather than loops over individual sites.

The central data structures are *masks*. For each direction and each of the four flanking groups, one mask records the corresponding bonds: bit b is set when bond b permits a hop in that direction and belongs to that group. There are therefore eight mask vectors in total. The simulation relies on three operations:

- **Counting** the allowed transitions in a group, as required for the tilted escape rate R_s in step 3 and the counting-field potential V_s in step 4, amounts to a population count over the words of its mask (`std::popcount`, one `POPCNT` instruction per word).



- | | | |
|---|--|---|
| ‡ | heap, uniformized

direct | a running bound on the weight spread exists at every instant, so the stop is the moment it reaches $\log(\text{target}_{\max})$

no such bound exists mid-block, so the stop is predicted in advance from an adaptive step τ |
| † | heap

uniformized

direct | every process on one global clock, events ordered by a binary heap; rejection-free

proposals generated at a configuration-independent maximal rate, rejected down to the true transition rate

one walker at a time, run alone from stop to stop; rejection-free |

Figure 5: The cloning loop of Section 1.4. The flow chart is common to the three event mechanisms; the two steps at which they differ are marked ‡ and †, as in the boxed procedure, and expanded below the chart, and the replay branch exists for the direct mechanism alone.

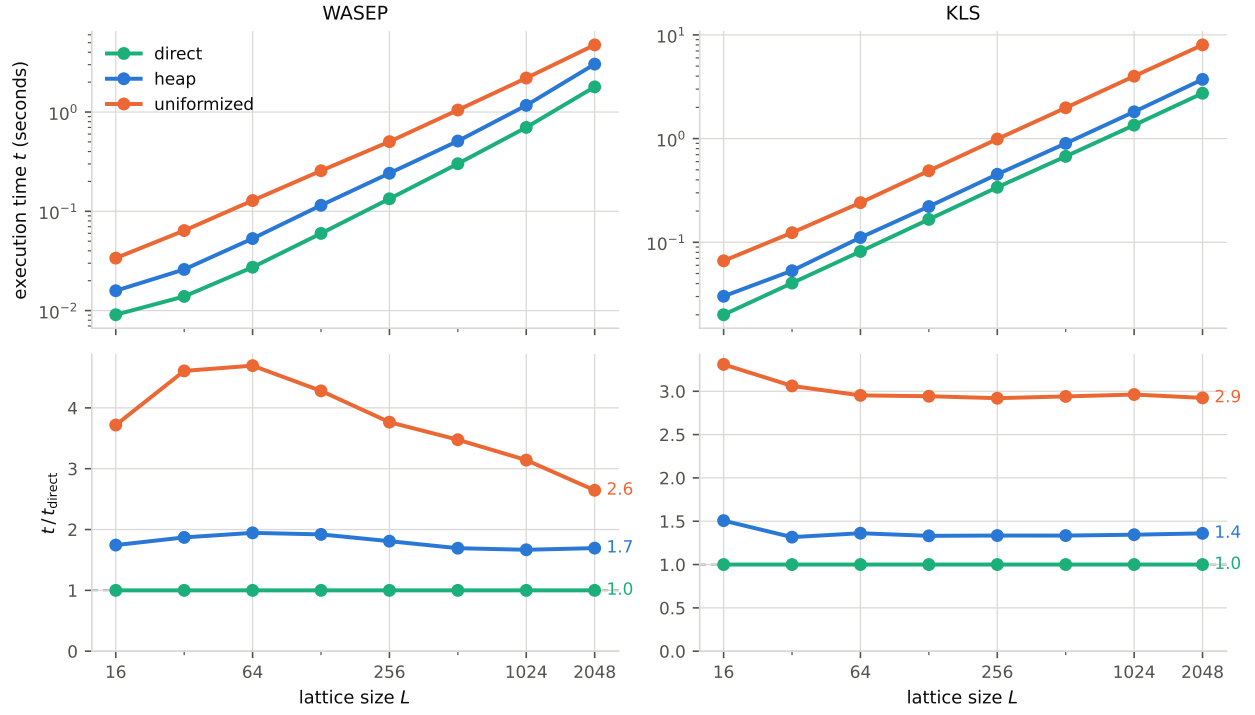
- **Selecting** the n -th allowed bond in a group, whenever step 3 selects a transition, is the most frequently executed individual operation in the simulation. Since all bonds in a group have the same rate, one uniform draw selects both the group and, by division, a uniform bond within it. Selecting the n -th bond then amounts to locating the n -th set bit of its mask. We consider two implementations of this operation. The portable version serves as a fallback on processors for which the optimized instruction is unavailable or inefficient. It clears the lowest set bit $n - 1$ times (`mask &= mask - 1`) and then counts trailing zeros. The optimized version uses the BMI2 instruction `PDEP` to select the n -th set bit directly; counting trailing zeros then gives the corresponding bond.
- **Updating** the masks after an event is a local operation. A hop changes the occupations of two neighboring sites, so only nearby bonds can gain or lose an allowed transition. Since the rate also depends on the sites beside the hopping pair, the change reaches one bond further in each direction. In total, only five bonds need to be reconsidered after each event, making the update $O(1)$ rather than $O(L)$.

In contrast to the interacting models, which update their masks locally by reconsidering only nearby bonds after each event, the uniform-rate WASEP implementation rebuilds its masks in full. Because all rates are the same, WASEP needs only two masks: one for right-moving hops and one for left-moving hops. The flanking occupations are irrelevant, and whether a bond permits a hop depends only on the occupations of its two sites. The masks can therefore be rebuilt one word at a time. Indeed, after shifting the configuration by one site to align n_{b+1} with n_b , the right- and left-moving masks are, respectively, `n & ~shifted` and `~n & shifted`. A single pass over $\lceil L/64 \rceil$ words is faster than regrouping five bonds across eight group vectors for lattices of up to a few thousand sites. Consequently, selecting the WASEP model explicitly is generally faster than representing the same dynamics as KLS with $\epsilon = 0$ and $\delta_{\text{KLS}} = 0$.

1.5.2 Scalability and tests

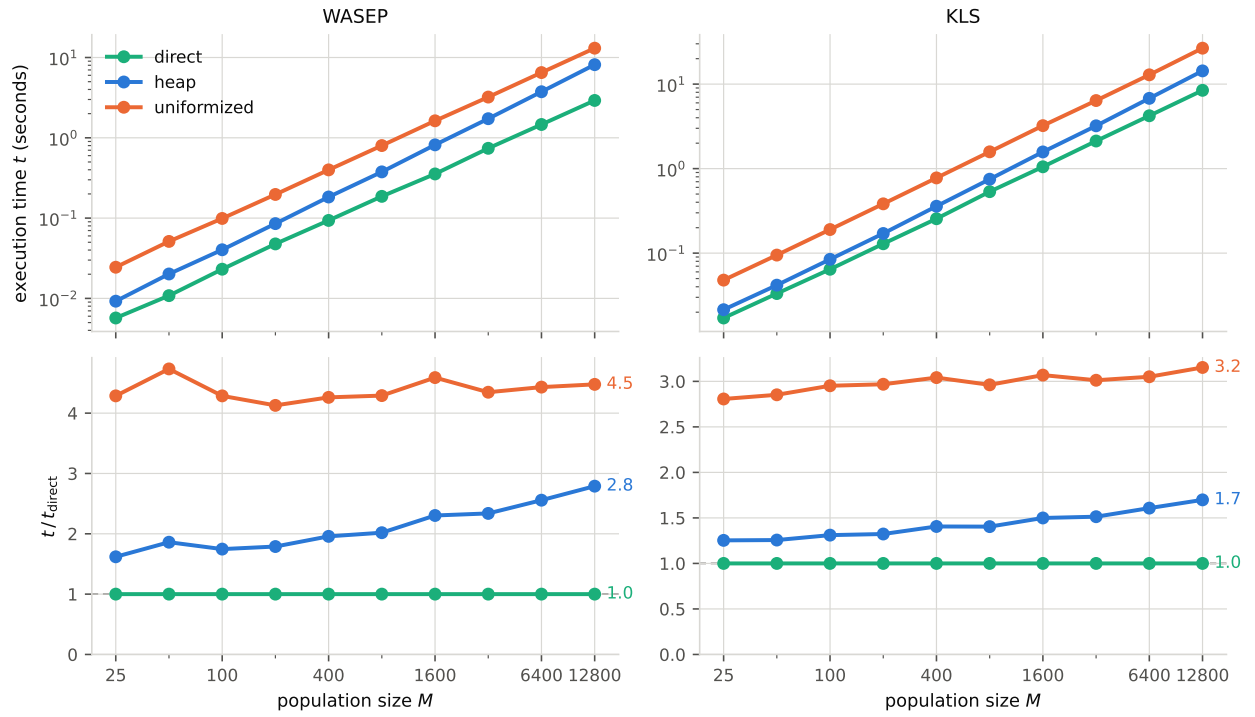
Figures 6 and 7 give the cost of the three different implementation mechanisms as the lattice and the population grow, for each rate model separately. Each point consists in one single run at fixed chain size, population and final time. Two properties of the implementation are visible in them. For KLS, the ratios are flat in the system size, as all operations are performed locally, see Section 1.5.1. For WASEP they drift downwards with L because, since the word-parallel rebuild costs $O(L/64)$ per event, the overhead of a mechanism becomes a smaller share of a rising per-event cost. Against M , the **Heap** penalty grows in both models, which is the $O(\log M)$ sift cost.

The engine is checked against two kinds of tests. In the first, the tilted generator is written out in full and diagonalised for a small open chain, with its dominant eigenvalue being the cumulant generating function exactly. The diagonalisation in `validation/exact/SSEP_TM.py` spreads the counting field over the bonds exactly as the engine does, so the two report the same observable and their numbers compare directly. Below, $\alpha = \beta = 1$ with $\gamma = \delta = 0$, holding the reservoirs at $\rho_L = 1$ and $\rho_R = 0$; the simulations use $M = 4000$ and $t_{\text{max}} = 4000$. No deviation exceeds 10^{-4} . That is the statistical error of the estimate and not a disagreement between the mechanisms.



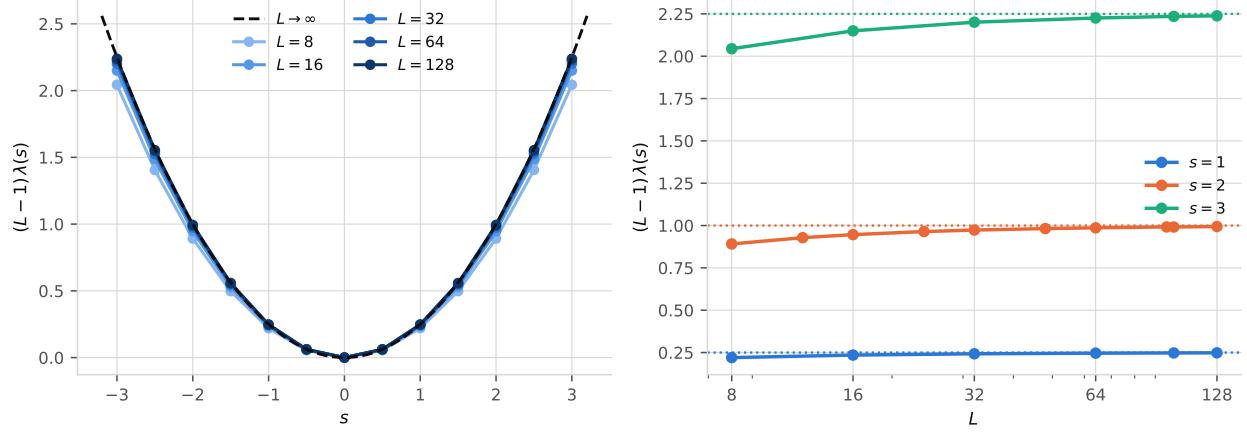
$M = 200$, $t = 50$, open boundaries, $E = 5$, $s = -1$, KLS at $\varepsilon = 0.6$. Median of repeated runs on Intel(R) Core(TM) i9-14900HX.

Figure 6: The three mechanisms against lattice size, at fixed population. Above, the execution time of one run; below, the same divided by **Direct** at the same size. Flat lower curves mean a per-event cost that does not grow with L .



$L = 100$, $t = 50$, open boundaries, $E = 5$, $s = -1$, KLS at $\epsilon = 0.6$. Median of repeated runs on Intel(R) Core(TM) i9-14900HX.

Figure 7: The same against population size, at fixed lattice. The rise of the **Heap** curve below is its $O(\log M)$ sift; the **Uniformized** one is flat, its rejections being a per-event cost.



Open SSEP, $E = 0$, equal reservoirs, $M = 2000$, $t = 2 \times 10^4$. Dotted lines are the limit at each s .

Figure 8: The open SSEP against its large- L closed form. Left, the rescaled generating function over the whole counting field at several sizes; right, three of those fields followed as the lattice grows, each limit dotted.

L	s	exact	direct	heap	uniformized
5	-1	-0.137095	-0.137048	-0.137058	-0.137000
5	+1	0.201977	0.201963	0.201945	0.201933
6	-1	-0.117955	-0.117939	-0.117980	-0.117915
6	+1	0.172403	0.172360	0.172368	0.172381
8	-1	-0.092207	-0.092177	-0.092170	-0.092181
8	+1	0.133351	0.133319	0.133341	0.133367

In the second test we benchmark the engine against the analytical results found in [2]. With no external field and the reservoirs holding equal densities, the open SSEP has a cumulant generating function known in closed form, but only to leading order in L . Plotting the rescaled CGF $(L-1)\lambda(s)$ for different values of L , we observe in figure 8 that the whole curve collapses onto the $L \rightarrow \infty$ analytical solution as the lattice grows. At fixed s the approach is monotonic from below.

1.6 Parameter reference

Table containing every quantity the engine takes, spelled as it is written in the plan file (".toml").

symbol	name in the plan	meaning
<i>Model and fields</i>		
L	length	number of sites
—	model	WASEP, KLS or NNN; selects where the eight rate-group weights come from
ϵ	epsilon	KLS coupling to the difference of the flanking occupations; 0 recovers WASEP
δ_{KLS}	delta_kls	KLS coupling to their sum; 0 preserves particle-hole symmetry.
$w_{\rightarrow}, w_{\leftarrow}$	right_weights, left_weights	the four class rates per direction, given explicitly for <code>model = "NNN"</code>
E	field	external driving field
s	bias	counting field
k	measurement_strength	monitoring coupling to the reference; $k = 0$ switches the reference off
<i>Boundaries</i>		
—	boundary	"open" or "periodic"
ρ	filling	periodic case: initial density
α, γ	alpha, gamma	open case: injection into, and removal from, the leftmost site
δ, β	delta, beta	open case: injection into, and removal from, the rightmost site
<i>Population and estimator</i>		
M	walkers	population size
—	target_min	weight ratio at which resampling becomes worthwhile
—	target_max	weight ratio that must never be exceeded
—	cloning_interval	optional extra resampling on a fixed clock
<i>Run</i>		
t_{max}	time	final time
—	dynamics	"direct", "heap" or "uniformized"; the event mechanism, described in the mechanism note of Section 1.4
—	initial	initial configuration: "empty", "alternating", "random" or "full". Open chains only; a ring takes its particle number from <code>filling</code>
—	seed	random seed; sweeping it gives independent repetitions
—	record_interval	time between recorded rows

References

- [1] B. Derrida, M. R. Evans, V. Hakim, and V. Pasquier. Exact solution of a 1d asymmetric exclusion model using a matrix formulation. *Journal of Physics A: Mathematical and General*, 26(7):1493–1517, 1993.

- [2] T. Bodineau and B. Derrida. Current fluctuations in nonequilibrium diffusive systems: an additivity principle. *Physical Review Letters*, 92(18):180601, 2004.
- [3] S. Katz, J. L. Lebowitz, and H. Spohn. Nonequilibrium steady states of stochastic lattice gas models of fast ionic conductors. *Journal of Statistical Physics*, 34(3):497–537, 1984.
- [4] Hugo Touchette. The large deviation approach to statistical mechanics. *Physics Reports*, 478(1–3):1–69, 2009.
- [5] Daniel T. Gillespie. A general method for numerically simulating the stochastic time evolution of coupled chemical reactions. *Journal of Computational Physics*, 22(4):403–434, 1976.
- [6] Daniel T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977.
- [7] Cristian Giardinà, Jorge Kurchan, and Luca Peliti. Direct evaluation of large-deviation functions. *Physical Review Letters*, 96(12):120603, 2006.
- [8] Vivien Lecomte and Julien Tailleur. A numerical approach to large deviations in continuous time. *Journal of Statistical Mechanics: Theory and Experiment*, 2007(03):P03004, 2007.
- [9] Augustine Kong, Jun S. Liu, and Wing Hung Wong. Sequential imputations and bayesian missing data problems. *Journal of the American Statistical Association*, 89(425):278–288, 1994.
- [10] Genshiro Kitagawa. Monte carlo filter and smoother for non-gaussian nonlinear state space models. *Journal of Computational and Graphical Statistics*, 5(1):1–25, 1996.
- [11] Randal Douc, Olivier Cappé, and Eric Moulines. Comparison of resampling schemes for particle filtering. In *Proceedings of the 4th International Symposium on Image and Signal Processing and Analysis (ISPA)*, pages 64–69, 2005.